

Support Vector Machines Intelligence

Linear SVM Implementation from Scratch

Instructions

- **Submit a single .py file** containing the complete implementation and visualization.
- **Do not plagiarize.** Geometric logic must be your own.
- **Naming convention:** YourName_BSAI2xxxx_Lab_SVM.py
- **IMPORTANT:** The use of `scikit-learn` or other ML libraries is **strictly forbidden**.
- You are allowed to use **NumPy** and **Matplotlib** only.

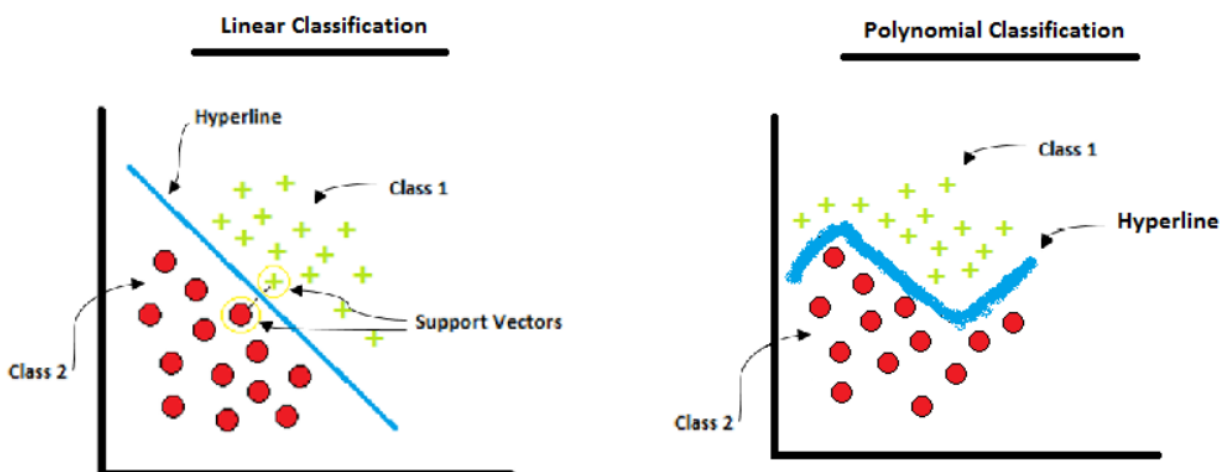
Background: Support Vector Machines

What is an SVM?

An SVM is a discriminative classifier formally defined by a separating hyperplane. Given labeled training data, the algorithm outputs an optimal hyperplane which categorizes new examples. In two-dimensional space, this hyperplane is a line dividing a plane into two parts where each class lies on either side.

The Maximum Margin Intuition

Unlike other classifiers, SVM does not just look for any line. It looks for the **Maximum Margin Hyperplane**. It maximizes the distance between the nearest points of each class. These closest points are called **Support Vectors**.



Linear vs. Polynomial (Non-Linear) Decision Boundaries.

Linear vs. Polynomial Classification

As shown in the above figure, Linear Classification (left) uses a straight hyperplane. This works perfectly when data is linearly separable. However, real-world data is often complex. Polynomial Classification (right) uses a non-linear boundary to separate between classes. In this lab, we focus on building the **Linear** version from scratch to understand how the weights are updated to maximize that margin.

The Math: Hinge Loss and Gradient Descent

To train the model, we minimize a Loss Function $J(w)$ that combines the margin width and classification error:

$$J(w) = \lambda \|w\|^2 + \frac{1}{n} \sum \max(0, 1 - y_i(w \cdot x_i - b))$$

The Mathematics of Optimization

We aim to minimize the following objective function (Hinge Loss + L2 Regularization):

$$J(w) = \lambda \|w\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i - b))$$

Where:

- $y_i \in \{-1, 1\}$ are the class labels.
- w is the weight vector (normal to the hyperplane).
- λ is the regularization parameter (trade-off between margin width and classification accuracy).
- η (learning rate) is the step size for gradient updates. Typical values range from 0.001 to 0.1.

Gradient Update Rules

For each sample x_i :

1. **If correctly classified outside the margin** ($y_i(w \cdot x_i - b) \geq 1$):

$$w = w - \eta \cdot (2\lambda w)$$

2. **If inside the margin or misclassified** ($y_i(w \cdot x_i - b) < 1$):

$$w = w - \eta \cdot (2\lambda w - y_i x_i), \quad b = b - \eta \cdot y_i$$

Lab Tasks

Task 1 - Data Synthesis and Preprocessing

Goal: Create a linearly separable environment.

- Use `np.random.randn` to create two clusters of points (e.g., centered at $[2, 2]$ and $[-2, -2]$).
- **Crucial:** Convert your labels to -1 and $+1$.
- Normalize your features manually using Standard Scaling (subtract mean, divide by standard deviation). Do not use `sklearn.preprocessing`.

Task 2 - The NumpySVM Class

Goal: Build the model architecture.

- Initialize w as an array of zeros and b as 0.
- Implement the `fit(X, y)` method containing the training loop.
- Implement the `predict(X)` method: $\text{sign}(w \cdot x - b)$.

Task 3 - Optimization via SGD

Goal: Apply the gradient update rules derived in the background.

- Use the following hyperparameters: learning rate $\eta = 0.01$, regularization $\lambda = 0.01$, epochs = 1000.
- Iterate through the dataset using stochastic gradient descent (update after each sample).
- In each iteration, check the condition $y_i(w \cdot x_i - b) \geq 1$.
- Update w and b based on whether the condition is met.

Task 4 - Geometry Visualization

Goal: Visualize the margin street.

- Plot the data points using different colors for each class.
- Draw the decision boundary: $w \cdot x - b = 0$.
- Draw the upper and lower margins: $w \cdot x - b = 1$ and $w \cdot x - b = -1$.
- Highlight the **Support Vectors** (points lying within or on the margin boundaries).

Task 5 - Experimental Analysis

- **Varying Lambda:** Train the model with $\lambda = 1.0$ and $\lambda = 0.001$. Explain how the margin width changes.
- **Linear Failure:** Attempt to train your SVM on a non-linear circles dataset (use `make_circles` from `sklearn.datasets` is allowed for data generation only). Document the accuracy and explain why a linear classifier fails.
- **Loss Convergence:** Plot the Hinge Loss over the number of epochs. Note that the loss may appear jagged; consider plotting a moving average or recording loss every 10-50 epochs.